

Technical Product Requirements Document

Flick: Development & Engineering Phase

Author: Product Designer & Engineer

Version: 1.2.0-en

Date: May 2026

1. Engineering Overview & Architecture

This document is the technical PRD for the development phase of **Flick**. The architectural goal is to build a lightweight client application as a **Single-Page Application (SPA)** with on-chain interactions delegated to third-party infrastructure layers (**Reown, Arc, and Goldsky**) in order to minimize computational burden on the user side.

The product centers on a simple creator flow:

- creators claim a nickname,
- receive a public tip page at `flick.to/[nickname]`,
- and manage analytics and withdrawal settings from a private dashboard.

The supporter experience must feel fast, tactile, and low-friction, matching the product name **Flick**.

2. Technology Stack Selection

2.1 Primary Stack

- **Framework:** Next.js 14 (React) + TypeScript

Used for dynamic routing (`flick.to/[nickname]`), strict typing, and performance-oriented rendering.

- **Styling & UI:** Tailwind CSS + Framer Motion

Tailwind is used for glassmorphism utility styling, while Framer Motion powers tactile swipe/flick interactions and UI animation.

- **Web3 Wallet Layer:** Reown AppKit + Wagmi

Provides ready-to-use authentication UI and hooks for wallet connection, social login, and basic smart contract interaction.

- **Execution Engine:** Arc Network SDK

Handles agentic workflows, including automatic stablecoin conversion and transaction execution orchestration.

- **Data Indexer:** Goldsky + GraphQL

Custom subgraphs and real-time pipelines index creator and tip activity for live frontend updates.

2.2 Architecture Intent

The application should offload as much complexity as possible from the frontend:

- **Reown** handles authentication and smart account flows.
- **Arc** handles execution logic and conversion workflows.
- **Goldsky** handles indexing, querying, and real-time event updates.

This keeps the client responsive while still delivering on-chain functionality.

3. Data Models & Subgraph Schema (Goldsky)

The following GraphQL schema will be used in Goldsky to index events emitted by the Flick smart contract. The structure is intentionally flat to keep frontend queries fast.

```
type Creator @entity {
  id: ID! # Creator wallet address
  nickname: String! @index
  totalTipsReceived: BigDecimal!
  createdAt: BigInt!
  tips: [Tip!]! @derivedFrom(field: "creator")
}

type Tip @entity {
  id: ID! # Transaction hash
  creator: Creator!
  senderAddress: Bytes!
  senderName: String # Derived from Reown social metadata
  amountUSDC: BigDecimal!
  message: String
  timestamp: BigInt!
}
```

4. Core Integrations & Endpoint Strategy

4.1 Reown AppKit Integration

Reown AppKit is initialized at the application provider/layout level. Configuration must explicitly enable:

- wallet connection,
- social login (email / Google, and optionally X),
- smart account creation,
- and session persistence.

This allows supporters to enter the flow with minimal setup friction.

4.2 Arc Network Execution Flow

When a supporter clicks or swipes the tip action, execution should be routed through the Arc Network agent layer. This agent handles the transaction workflow, including optional token conversion logic when a supporter is not already paying in the target stablecoin.

4.3 Real-Time Data Updates

The frontend subscribes to live updates through Apollo Client and Goldsky subscription infrastructure so the activity feed can update without a refresh.

5. UI/UX & State Management Implementation

The interface must feel immediate, tactile, and premium.

5.1 Interaction Principles

- **Flick / Swipe interactions:** Use gesture-driven motion on the final tip-confirmation action.
- **Fast visual feedback:** Pending states, button progress, and optimistic UI should make the system feel responsive.
- **Low-friction onboarding:** Creator setup should avoid long forms and instead rely on a short guided transition.
- **Supporter confidence:** The tip action should clearly show recipient identity, amount, and confirmation state.

5.2 Visual Style Constraints

The main creator card should use a glassmorphism-style treatment such as:

- ``bg-white/30``
- ``backdrop-blur-md``
- ``border border-white/40``
- ``shadow-xl``
- ``rounded-3xl``

6. Screen-by-Screen Product Specification

Added from screenshot reference and normalized into the React/Next.js implementation plan.

6.1 Screen 1 — Landing & Pitch Page

Route: /

The main landing page is designed to attract creators and explain the product value proposition.

Requirements:

- **Hero Section:** Minimal typography animation communicating the core message: **“Receive Tips in a Flick.”**
- **Primary CTA:** A **“Create Your Tip Jar”** button that opens the Reown AppKit modal.
- The modal flow should support:
 - wallet connection,
 - email registration,
 - Google sign-in.
- The page should emphasize simplicity, speed, and creator-first onboarding.

6.2 Screen 2 — Creator Onboarding / Claim Nickname

Route: `/claim`

Visibility: Appears after first login.

This screen should act as a clean one-step transition page, without long forms.

Requirements:

- **Input UI:** A single text field showing the structure `‘flick.to/[nickname]’` while the user types.
- **Live Validation:** Nickname availability must be checked in real time via a Goldsky-backed lookup layer.
- **Primary CTA:** **“Claim & Launch”** button.
- Clicking the CTA triggers the smart contract execution flow through Arc Network.
- The user should clearly understand that claiming the nickname creates their public tipping identity.

6.3 Screen 3 — Public Tip Jar Card (Core Experience)

Route: `/:nickname`

Example: `/aldidesign`

This is the main public page where fans send tips.

Requirements:

- **Card Header:** Show creator avatar, display name, and connection or verification status when available.
- **Interaction Area:** Include a tip amount input and an optional message field.
- **Swipe/Flick Button:** Use a gesture-driven payment confirmation interaction.
- **Activity Feed:** Show recent support activity at the bottom of the card.
- Activity updates should appear instantly using real-time Goldsky GraphQL subscriptions and reactive client-side state.
- The page styling should strongly reflect the glassmorphism design language.

6.4 Screen 4 — Creator Dashboard (Private Settings)

Route: /dashboard

This is a private creator page for analytics and account settings.

Requirements:

- **Overview Panel:** Display total tips received in USDC, with an optional mini-chart.
- **Quick Copy:** Provide a one-click copy action for the public profile link (`flick.to/nickname`).
- **Withdrawal Config:** Show the destination wallet address managed by the Reown smart account.
- The dashboard should prioritize clarity and fast access to actionable creator controls.

6.5 Implementation Mapping Note

The screenshot reference uses a few terms that should be mapped into the current stack as follows:

- **“Angular Animations”** → implement with **Framer Motion** in Next.js/React.
- **“Signals”** → implement with **reactive client state** plus Apollo subscription updates.
- **“Goldskey RPC”** → implement as a **Goldskey-backed nickname availability check**, or equivalent indexed lookup/query service.

These wording differences should not change the architectural direction defined in this PRD.

7. Engineering Task List (Phase-by-Phase)

Phase 1 — Project Setup & UI Foundation

- Initialize the Next.js 14 repository with TypeScript and Tailwind CSS.
- Set up Framer Motion base configuration for swipe/flick gestures.
- Build static UI components:
 - Glassmorphism Card
 - Tip Amount Input
 - Optional Message Input
 - Activity Feed UI
 - Landing Hero / CTA block
- Implement dynamic routing for `flick.to/[nickname]`.
- Add dedicated routes for `/claim` and `/dashboard`.

Phase 2 — Wallet Connectivity (Reown Layer)

- Integrate Reown AppKit at the provider level (`layout.tsx` or equivalent app root).
- Configure social login methods (email, Google, optionally X).
- Enable background smart account creation.
- Implement the creator registration flow for nickname claiming.

- Test creator wallet and supporter wallet flows in a testnet environment.

Phase 3 — Execution & Agentic Workflows (Arc Layer)

- Integrate Arc Network SDK into the **Send Tip** / **Flick to Confirm** action.
- Configure the execution agent to support Stablecoin FX / auto-swap behavior.
- Implement pending, success, and failure transaction states in the UI.
- Connect the **Claim & Launch** flow to smart contract execution.

Phase 4 — Real-Time Indexing (Goldsky Layer)

- Write and deploy the custom subgraph using the `Creator`` and `Tip`` entities.
- Set up Apollo Client in Next.js for WebSocket or equivalent real-time connectivity.
- Connect GraphQL subscription data to the Activity Feed so support history appears instantly.
- Implement nickname availability checks using indexed data or a Goldsky-backed lookup path.

Phase 5 — Polish, Security & Deployment

- Apply input sanitization / XSS filtering to supporter messages.
- Optimize rendering latency and validate slippage protection boundaries in Arc flows.
- Audit mobile UX transitions, especially the flick interaction.
- Validate route protection for `/claim`` and `/dashboard``.
- Deploy the frontend to Vercel and connect the `flick.to`` domain.

8. Delivery Notes

This English version preserves the original technical direction while incorporating the screenshot-based screen requirements into the core PRD.

Primary product flow:

1. Creator lands on `/``
2. Opens Reown modal from the CTA
3. Logs in and proceeds to `/claim``
4. Claims nickname through Arc-backed contract flow
5. Receives public page at `/:nickname``
6. Monitors account performance from `/dashboard``

This flow should serve as the canonical implementation path for the development phase.